Quantum Railways · Participant guide

FreeYaMind Campus · Beginner challenge

Minimum path: start here

1. **What do I open?** Read README.md, then upload **01_quantum_railways_lab.ipynb** to Colab or open it locally.
2. **What must I complete?** Exercises A–C, read/run the supplied QUBO-to-Hamiltonian conversion, complete F/G, train locally, then bind the final angles, compile once and sample a local noisy/synthetic backend. Allow **about 3 hours** for this core. D/E derivations and broader hardware work are optional and take extra time.
3. **What can I change?** After the core, choose one question in the final **Extension workspace**. Its explicit route/budget/energy inputs keep your experiment separate from the baseline.
4. **What do I submit?** Your completed lab or extension notebook, relevant results/visuals and an explanation using SUBMISSION_TEMPLATE.md.
5. **What counts as success?** Genuine understanding, a correct quantum component, a fair classical comparison and insight. A carefully explained failure is valuable; the highest railway score is not the goal.

The game and the learning goal

Build fictional connections between Romanian cities, including Iași, București, Cluj-Napoca, Timișoara, Brașov, Sibiu, Craiova and Constanța. Each route has an integer cost and value. Your budget cannot buy everything. The lab first uses just five route decisions; you can later investigate a different objective, representation or quantum method.

> This is a fictionalized optimization map created for the hackathon. Costs, routes, and rewards do not represent actual Romanian railway infrastructure or policy recommendations.

A route is one edge between two cities; a path may use several routes. Edges are undirected and bought once. The introductory true score is the sum of built route values. Actual construction cost must be at most the budget. Disconnected tracks and the empty plan are allowed. Destination tickets are an optional extension, not a hidden scoring rule in the lab. City locations are geographic; connections, costs and values are fictional. The notebook embeds longitude/latitude and all route data directly, without fetching map files.

You will learn to encode plans as bitstrings, evaluate cost/value/validity, construct an energy model, audit a QUBO and Hamiltonian, build QAOA layers, optimise parameters locally, and compare samples with classical references. You will also investigate hardware mapping and a final-circuit sampling workflow.

Core path in the notebook

Read README.md for upload and local setup instructions. Open **01_quantum_railways_lab.ipynb** in Colab or your local notebook interface. The notebook is self-contained: no local modules, images, JSON inputs, repository cloning or Drive mounting are required.

The setup cell installs only missing or mismatched exact versions and checks imports. Restart if asked. Keep the tested environment until you have a reason and time to validate a change. A CPU runtime is sufficient; a GPU is unnecessary for five qubits. The setup installs only the tested computing packages; notebook-building tools are separate organiser dependencies.

Plan your time

Allow **about 3 hours** for the core, including coding and discussion: roughly an hour for the game and A–C, an hour for the supplied conversion and F/G, and an hour for optimisation, sampling and interpretation. Some teams will need longer; this is learner time, not notebook execution time.

Optional maths: derive one linear coefficient and one interaction, then turn on `MATHEMATICAL_DEEP_DIVE` for full D/E arrays. **Optional hardware study:** use the appendix after the conclusion to compare levels/seeds or investigate mapping and recovery. Real QPU access and queues take additional time. Ask a mentor for a hint rather than copying an answer.

The guided lab workflow

Complete the selected routes, cost, value and feasibility functions first. Cost and value are different fields; a boundary plan spending exactly the budget is valid. Variable position refers to the five-route decision list, not automatically the same physical

route ID.

Then implement the energy and compare every one of the 32 plans. Inspect both the exact energy minimum and the best feasible true-score plan. Ask two separate questions: **did the optimiser minimise the encoded energy?** and **did energy encode the intended planning rule?** Discover the numerical result yourself before forming a conclusion.

The supplied QUBO and Ising conversion keeps the model visible on the core path; deriving all coefficients in D/E is optional. The cost-layer check compares phases against classical energy, including an irrelevant global phase. QAOA alternates cost and mixer operations; interference reshapes probabilities while a classical optimiser updates angles. It is not simple relaxation into a lowest-energy state, and does not guarantee the optimum.

Training in the default path uses exact ideal statevector expectations. The final ideal and backend circuits use finite shots. Keep that distinction in your explanation. Qiskit prints variable 0 on the right; decode the named logical `plan` register, not a physical-qubit list.

You are given plotting primitives and result tables, not every final argument or plot. Choose a visualization that answers a question. Use teal for valid results, coral for invalid results, gold for cost/reward, navy for quantum/model concepts and grey for unselected routes.

Hardware reference (optional live access)

The core is **local optimisation → bind final angles → compile once → local noisy sampling**. It needs neither IBM Runtime nor credentials. Compare **P(best feasible)** (including ties), **P(feasible)** and **expected energy**, alongside compiled depth and two-qubit count. The synthetic device has generated noise, not a current hardware calibration.

Optional real QPU

With organiser-approved access, enable `USE_REAL_HARDWARE` and use an assigned `BACKEND_NAME`; an empty name deliberately enables discovery. The optional cells compile the same fixed circuit for that device, submit with Sampler, print its backend/job ID, then retrieve counts. Copy/save the ID immediately. A real-access failure is reported rather than silently replaced by a local result. The core results remain available.

The real path uses the client-side `Sampler` and `channel='ibm_quantum_platform'`. It requires a separately organiser-validated IBM environment; the core pins do not install a live IBM client. Hosted Colab and live IBM execution still require organiser smoke tests. The default lab never submits the parameter-optimisation loop to a QPU.

Credential safety

Use Colab Secrets under `IBM_QUANTUM_TOKEN` or the supplied masked prompt. Never paste a token into source, share it, commit account files or include sensitive outputs in a submission. The notebook never saves accounts. Clear sensitive outputs before sharing.

Record the purpose and configuration of each real experiment, including its backend/job ID, shots, angles and compilation settings. Use your allocated QPU budget for meaningful experiments and batch related parameter values when practical; follow organiser access instructions.

Optional appendix: deeper hardware questions

After the main conclusion, the appendix provides compilation sweeps, calibration inspection, parameterized compilation, optional JSON exports and recovery by job ID. None is required for the core. Level 3 is not automatically best; the routing delta is a comparison with an all-to-all compilation, not a literal SWAP count.

The parameterized helper matches the ISA circuit's actual parameter order. Its optional job-record helper writes IDs before waiting for results. `SAVE_RESULTS` exports `railway_results.json` only when enabled; download saved files before closing a temporary Colab session. If a real job was already submitted, retrieve it rather than submit a duplicate.

After the lab: one open-ended question

Follow this small research workflow:

1. State a question and prediction.
2. Define the unchanged baseline and true rules.
3. Make one meaningful change.

4. Design an experiment with a fair reference.
5. Collect numerical or visual evidence.
6. Explain what you learned, including uncertainty and failure.

Use the notebook's final **Extension workspace** to write your question, prediction, unchanged baseline, change, classical reference and evidence. Start from `extension_routes`, `extension_budget`, `extension_penalty` and `extension_depth`. `enumerate_plans(routes, budget, energy_function)` builds a fresh reference; `distribution_metrics(counts, routes, budget, reference_rows)` uses that reference rather than the demo's globals; `build_qaoa(qubo, p)` accepts your model/depth. Recompute references when the energy changes. For a new objective, update the QUBO and true scoring rule as needed, not only a label.

A complete extension project changes something meaningful, retains an objective-dependent parameterized quantum component, compares with a classical reference, presents evidence and explains limitations. It can combine modelling, QAOA and an engaging interface; advanced quantum research is not required.

A beginner team may instead complete and explain the guided lab. Demonstrate your own understanding: show predictions, completed functions, validated results and interpretation rather than submitting only executed cells. An extension offers room for originality, not a prerequisite for asking useful questions.

Extension idea bank — questions, not a feature checklist

Direction	A question you might investigate
Destination tickets	Can you reward a destination only when purchased routes create a path? How would you detect/encode connectivity; are auxiliary bits needed, and what changes?
Two players	What happens if routes can be shared, claimed exclusively or blocked? What does each player optimise, and can a small binary model express it?
Multiple objectives	How should value, cost, coverage, connectivity or resilience compete? Does one weighted term dominate the model?
Robustness	Is a high-scoring network still useful after a route fails or a cost changes? Can alternative paths be rewarded?
Quantum workflow	How do depth, mixers, initialisation, optimiser or training/sampling choices affect the distribution? What changes with hardware mapping, noise or supported error suppression?
Experience	Can an interactive planner, board-game interface, parameter explorer or hardware/simulator visualization explain why the model or circuit succeeds or fails?

These are examples, not a closed list. A graphical interface alone is insufficient unless it supports a meaningful model or quantum investigation.

After the baseline hardware experiment you might compare depth, optimisation levels, transpiler seeds, layouts, physical qubits or shots; investigate noisy simulation, supported dynamical decoupling/twirling, hardware-in-the-loop training, or QAOA-aware routing. Start with one question. Local testing does not necessarily implement real-device suppression options; check the tested SDK/backend support before interpreting them.

Advanced experimental guidance (optional reference)

Keep **energy**, **true score**, **feasibility**, **best feasible sample** and **most frequent sample** distinct. A high-scoring invalid plan is not success. A lowered energy is not proof that the planning model improved. A best sample can hide a poor distribution.

Exact enumeration is an appropriate reference for five bits and is faster than QAOA simulation here. It scales as 2^n, so keep early variants small. Compare the same decision space and rules; an unrestricted physical-network reference answers a broader question than a restricted representation. Uniform sampling is also informative when there are few possible plans. No quantum advantage is claimed.

Use multiple seeds (at least three for a stochastic extension) or comparable repeated scenarios. Keep shots/training budgets comparable and preserve individual runs. Explain a changed budget rather than silently favouring one method.

Report feasible rate, target/model-minimum probability, optimal-feasible probability where known, expected energy, mean or median true score, best feasible score observed, and over-budget rate as applicable. State whether invalid outcomes are included in score averages. Record qubits, compiled depth, two-qubit gates, optimiser evaluations, training/final shots, runtime, versions and configuration. Simulator times are not hardware times.

A carefully investigated negative result or modelling failure is valuable. What evidence distinguishes a model problem from a parameter-optimisation, sampling, mapping or noise problem?

Submission and judging

Use **SUBMISSION_TEMPLATE.md** or the same structure in your notebook. Submit your completed lab or extension notebook, relevant results/figures, and enough configuration to reproduce the comparison. Explain the question, baseline, change if made, quantum role, classical reference, experiment, results, insight, limitation and next step. Include ordinary hardware job IDs and configurations for any real jobs; label fake/simulator results clearly. Clear secrets and sensitive outputs first.

A clear map, simple plots, an educational explanation or a thoughtful interactive demonstration can all communicate evidence. Railway score is not the main goal. Judging considers question quality/originality, correct quantum implementation and understanding, fair experimentation, insight from successes/failures, communication and reproducibility. A polished interface cannot replace the quantum component; simple visuals can support a strong investigation.

Proposed weights, awaiting organiser confirmation: proposal/question 25%, quantum correctness/understanding 25%, evidence/comparison/insight 20%, creative communication 20%, reproducibility 10%. These are not yet an official judging rubric. For guided-lab submissions, show genuine understanding through completed checks, reasoning and evidence.

Troubleshooting

- **A TODO check stops:** read the exercise's context and expandable hints, test one small case, then ask a mentor. Required functions are intentionally incomplete; there is no hidden automatic solution.
- **Version mismatch or restart message:** restart the runtime and rerun setup. Do not repeatedly install packages in later cells. If internet installation is unavailable, use an organiser-prepared tested environment.
- **Wrong local kernel:** inspect `sys.executable`; choose the environment where you installed requirements. Install `ipykernel` separately if your interface needs a kernel; VS Code/Jupyter supplies the interface.
- **Edited a function:** rerun its checks and all dependent cells. Restart and rerun from the top if results seem stale.
- **Unexpected bits:** distinguish plan-list order from printed key order, and logical measurement registers from physical layout. The primitive decoder requires the named `plan` register.
- **IBM connection unavailable:** use the labelled local path and ask a mentor about instance/backend access. Do not paste a token into an error report. An assigned unavailable backend is not silently replaced by another real QPU.
- **A real job already submitted:** keep the saved ID, retrieve that job, and avoid accidental duplicate submissions.
- **Slow experiment:** reduce decision count/depth/training budget first. Hardware queue time differs from circuit execution time.

References

- IBM Quantum Learning
- Qiskit bit ordering
- IBM client-side Sampler migration
- IBM local testing mode
- Original QAOA paper